1. Webhooks & Event Ingestion

- **Stripe Webhooks Documentation**
    - **Link:** https://docs.stripe.com/webhooks?
    - **Topic Covered:** The architectural flow of event delivery, registering destination endpoints, and structuring JSON payload events.
    - **Application:** Provides the industry-standard blueprint for building the Endpoint and Event ingestion logic required in Steps 1 and 2.
- **Webhook.site**
    - **Link:** https://webhook.site/?utm/view/[redacted]25c6-71d8-4dc1-b187-ae34e83f7364#!/view/b99125c6-71d8-4dc1-b187-ae34e83f7364
    Text
    - **Topic Covered:** Inspecting webhook requests, viewing incoming HTTP requests, and understanding webhook payloads in practice.
    - **Application:** Open the link, copy the unique URL shown on the page, and use it as the webhook URL when sending a test request. Send the request, then return to Webhook.site to see the request appear. You can click it to view the data that was sent, including the JSON body and other request details.
- **YouTube:**
    - How WebHook works | System Design
    - Top 3 Things You Should Know About Webhooks!
    - APIs vs Webhooks

2. Idempotency & Duplicate Prevention

- **Stripe API Reference: Idempotent Requests**
    - **Link:** https://docs.stripe.com/api/idempotent_requests?
    - **Topic Covered:** Utilising a unique Idempotency-Key header to safely retry requests without accidentally executing the same operation twice.
    - **Application:** Explains the core theory required in Step 3 to ensure duplicate events sent by a client are intercepted and handled gracefully.
- **MongoDB Database Manual: Unique Indexes**

    - **Link:** https://www.mongodb.com/docs/manual/core/index-unique/
    - **Topic Covered:** Ensuring indexed fields do not store duplicate values and using compound unique indexes to enforce uniqueness across a combination of fields.
    - **Application:** Details the database-level implementation for the { userId: 1, idempotencyKey: 1 } index required to prevent duplicate events and handle the 11000 duplicate-key error.

- **YouTube:**
    - Idempotency - What it is and How to Implement it
    - Idempotency in APIs: you should be aware of this!
    - Idempotency in APIs Explained | Why It Matters + Code Example

3. Rate Limiting Algorithms

- **MDN: 429 Too Many Requests**
    - **Link:** https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/429
    - **Topic Covered:** The HTTP 429 Too Many Requests status code and the use of the Retry-After response header.
    - **Application:** Defines the HTTP response that Hookshot must return when a user exceeds the configured request limit in Steps 4 and 5.
- **Cloudflare: Rate Limiting Rules**
    - **Link:** https://developers.cloudflare.com/waf/rate-limiting-rules/?
    - **Topic Covered:** How rate limiting is used to control excessive traffic and protect web applications and APIs.
    - **Application:** Provides a real-world example of how rate limiting is used in production systems to protect APIs from excessive requests.
- **ByteByteGo: Design a Rate Limiter**
    - **Link:** https://bytebytego.com/courses/system-design-interview/design-a-rate-limiter?
    - **Topic Covered:** A visual breakdown of API rate limiting, including window-based algorithms and controlling excessive traffic.
    - **Application:** Serves as the theoretical foundation for building the fixed-window, in-memory Map rate limiter required in Steps 4 and 5.

- **YouTube:**
    - ▶ Rate Limiter System Design: Token Bucket, Leaky Bucket, Scaling
    - ▶ Five Rate Limiting Algorithms ~ Key Concepts in System Design
    - ▶ 🔥 How Rate Limiting and Throttling Saves Your API Server From CRASHING!

4. Caching & Invalidation

- **MDN: JavaScript Map**
    - **Link:** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map
    - **Topic Covered:** Using JavaScript's Map data structure to store and retrieve key-value pairs in memory.
    - **Application:** Provides the foundation for implementing Hookshot's in-memory cache, as required in Step 6.
- **Node.js: setTimeout()**
    - **Link:** https://nodejs.org/api/timers.html?#settimeoutcallback-delay-args
    - **Topic Covered:** Scheduling a function to execute after a specified delay.
    - **Application:** Can be used to implement the TTL mechanism that automatically removes cached endpoint data after the required 5-minute period.
- **GeeksforGeeks: Cache-Aside Pattern**
    - **Link:** https://www.geeksforgeeks.org/system-design/cache-aside-pattern/
    - **Topic Covered:** The lazy-loading flow of checking an in-memory cache first, querying the primary database on a cache miss, and storing the result back in the cache.
    - **Application:** Maps directly to Step 6, where getActiveEndpoints() should check the cache first and query MongoDB only when the data is not already cached.
- **YouTube:**
    - ▶ REST API Caching Strategies Every Developer Must Know
    - ▶ Caching Pitfalls Every Developer Should Know
    - ▶ Cache invalidation isn't a hard problem